Assignment 1 Report

Movie Reviews Sentiment Analysis Using Natural Language Processing (NLP)

Course: Natural Language Processing
Assignment: 1 – Dataset Exploration and Text Preprocessing
Project Topic: Movie Reviews Sentiment Analysis
Dataset: IMDB Movie Reviews Dataset

Team Members

1. **Imrah Maqsood** — Roll No: 0293
2. **Ajwa**

Submission Date: 8 November 2026

---

1. Introduction

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to understand and process human language.

The purpose of this project is to prepare movie reviews for sentiment analysis. Sentiment analysis identifies whether a review expresses a positive or negative opinion.

In this assignment, we explore a movie review dataset, examine its text characteristics, compare tokenization methods, and implement a reusable text-cleaning pipeline.

2. Dataset and Environment

2.1 Dataset Information

We used the IMDB Movie Reviews Dataset.

- Dataset source: https://zenodo.org/records/7928582
- Total documents: 50,000
- Positive reviews: 25,000
- Negative reviews: 25,000
- Columns: review and sentiment
- File format: CSV
- Licence: Not specified in the referenced Zenodo record.

The dataset contains movie reviews labelled as positive or negative. The raw dataset is stored locally and is not included in the Git repository.

2.2 Environment and Libraries

The project was developed using Python in Visual Studio Code.

Libraries used:

- Pandas: Loading and processing CSV data.
- NLTK: Tokenization and Porter stemming.
- re: Regular expressions and text pattern matching.
- unicodedata: Unicode normalization.
- pathlib: Managing file paths.

Dependencies are listed in `requirements.txt`.

2.3 Dataset Loading

The dataset was loaded using Pandas. The loading script reads the raw CSV file, prepares the dataset, and saves the processed dataset.

The script is available at:

`src/download_dataset.py`

3. Dataset Exploration

3.1 Basic Statistics

The following statistics were calculated:

Measurement	Result
Total documents	50,000
Average review length	231.16 words
Shortest review	4 words
Longest review	2,470 words
Positive reviews	25,000
Negative reviews	25,000

3.2 Target Distribution

The dataset has a balanced sentiment distribution.

Sentiment	Number of Reviews
Positive	25,000
Negative	25,000

Each class represents 50% of the dataset. This balance is useful because neither sentiment class dominates the dataset.

3.3 Vocabulary Analysis

Vocabulary analysis was performed on the first 1,000 reviews using word extraction.

Measurement	Result
Sample documents	1,000
Total tokens	239,546
Unique words (types)	17,960
Type-token ratio	0.075

The type-token ratio was calculated by dividing the number of unique words by the total number of tokens.

A low ratio indicates that many words occur repeatedly in the reviews.

3.4 Zipf's Law and Long-Tail Distribution

The most frequent words include:

Word	Frequency
the	13,431
and	6,474
a	6,385
of	5,911
to	5,319
br	4,124
is	4,069
in	3,727
it	3,668

The results show that a small number of words occur very frequently, while many other words occur less frequently.

This pattern is related to Zipf's Law and the long-tail distribution of natural language.

The word `br` appears because HTML line-break tags such as `<br />` are present in the original reviews.

4. Tokenization and Regular Expressions

4.1 Tokenization Methods

Two tokenization methods were compared using the first 1,000 reviews of the IMDB dataset.

Method 1: Custom Regex Tokenizer

A custom regular expression tokenizer was implemented to identify words, numbers, URLs, and HTML tags. It also keeps contractions and hyphenated words together as tokens.

Method 2: NLTK Treebank Tokenizer

The NLTK Treebank Word Tokenizer was used to split text into tokens. It separates punctuation and handles contractions according to Treebank tokenization rules.

4.2 Tokenization Comparison

The following results were obtained from the first 1,000 reviews:

Measurement	Regex Tokenizer	NLTK Treebank Tokenizer
Total tokens	233,440	270,980
Unique tokens	22,193	25,325
Type-token ratio	0.0951	0.0935

The Type-Token Ratio (TTR) was calculated by dividing the number of unique tokens by the total number of tokens.

The NLTK tokenizer produced more tokens because it separates punctuation and some contractions into individual tokens.

For example:

- Regex: `you'll`
- NLTK: `you`, `'ll`

4.3 Dataset-Specific Regular Expression Patterns

The following patterns were used to identify common structures in the dataset:

1. **HTML Tags:** `<[^>]+>` identifies HTML tags such as `<br />`.
2. **URLs:** `https?://\S+|www\.\S+` identifies web links.
3. **Rating Expressions:** `\b\d+(?:\.\d+)?\s*/\s*10\b` identifies ratings such as `8/10`.
4. **Hyphenated Words:** `\b\w+(?:-\w+)+\b` identifies words such as `light-hearted`.

In the first 1,000 reviews, the following patterns were counted:

Pattern	Count
HTML break tags	4,124
Rating expressions	75
URLs	3

Pattern	Count
Hyphenated words	1,523

4.4 Dataset-Specific Tokenization Challenges

Two important challenges were identified:

1. **HTML Tags:** Reviews contain HTML tags such as `<br />`. These tags can produce unwanted tokens if they are not handled properly.
2. **Contractions and Hyphenated Words:** Expressions such as `you'll` and `light-hearted` can be split differently by different tokenizers.

4.5 Conclusion

The comparison shows that tokenization methods produce different results depending on how they handle punctuation, contractions, HTML tags, and hyphenated words. The custom Regex tokenizer and NLTK Treebank tokenizer both provide useful approaches for preparing movie reviews for sentiment analysis.

5. Text Cleaning Pipeline

A reusable text-cleaning function was implemented in:

`src/clean_text.py`

The pipeline contains the following steps:

1. Unicode Normalization

NFKC normalization is applied to standardize Unicode characters.

2. HTML Tag Removal

HTML tags are removed using regular expressions.

3. Lowercase Conversion

All text is converted to lowercase so that words with different capitalization are treated consistently.

4. Noise Removal and Contraction Handling

URLs and rating expressions are removed. Common contractions are expanded before tokenization. For example, `you'll` is expanded to `you will`.

5. Tokenization

Words and numbers are extracted using a regular expression.

6. Stopword Removal

Common words that provide limited information are removed using a manually defined stopword list.

Negation words such as `not` and `no` are preserved because they can change the meaning of a movie review.

7. Stemming

Porter Stemmer from NLTK is used to reduce words to their stems.

For example, words such as `watching` and `watched` may be reduced to a common stem.

5.1 Cleaning Function

The cleaning function performs all preprocessing operations and returns cleaned text as a single string.

It is reusable because the same function can be applied to every review in the dataset.

5.2 Vocabulary Before and After Cleaning

Vocabulary was measured across the full dataset before and after cleaning.

Measurement	Vocabulary Size
Before cleaning	100,678
After cleaning	70,147
Vocabulary reduction	30,531
Percentage reduction	Approximately 30.3%

The vocabulary decreased because HTML tags, URLs, rating expressions, common stopwords, and different word forms were handled during preprocessing.

5.3 Example of Cleaning

Original review:

"One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked."

Cleaned review:

```
one other review mention after watch just 1 oz episod will hook
```

The cleaned text contains lowercase tokens, expanded contractions, fewer unnecessary words, and stemmed word forms.

6. Output and Reproducibility

The cleaned dataset was successfully saved as:

`data/processed/cleaned_reviews.csv`

The output contains the original review, sentiment label, and an additional `cleaned_review` column.

The project uses separate scripts for dataset preparation, tokenization comparison, and text cleaning.

To reproduce the process:

1. Install Python.
2. Create and activate a virtual environment.
3. Install dependencies using `pip install -r requirements.txt`.
4. Place the original dataset file named `IMDB Dataset.csv` in the main project folder.
5. Run `python src/download_dataset.py`.
6. Run `python src/tokenization.py`.
7. Run `python src/clean_text.py`.

The scripts generate the prepared and cleaned datasets and display tokenization comparison results.

7. Conclusion

In this assignment, we explored the IMDB movie review dataset and implemented a text preprocessing pipeline.

We examined dataset statistics, sentiment distribution, vocabulary size, and tokenization methods. We also identified common text problems such as HTML tags, contractions, URLs, ratings, and hyphenated words.

The cleaning pipeline applies Unicode normalization, noise removal, contraction handling, lowercase conversion, tokenization, stopword removal, and Porter stemming.

The vocabulary decreased from 100,678 to 70,147 unique words after cleaning, representing approximately a 30.3% reduction.

The cleaned dataset is now prepared for subsequent sentiment analysis tasks.

8. Team Contribution Statement

This project was completed collaboratively by two team members.

- **Imrah Maqsood (Roll No: 0293):** Participated in dataset exploration, tokenization comparison, text preprocessing, implementation, testing, and report preparation.
- **Ajwa:** Participated in project development, dataset exploration, text preprocessing, testing, and documentation.

Both team members worked together on the assignment and contributed to completing the project deliverables.